

□ LI - Encapsulation and Polymorphism

□ Question - 1:

What is encapsulation? Explain how java achieves it using access modifiers and getter/setter methods. Give one real-life analogy.

Answer:

Encapsulation is the process of wrapping data (variables) and methods into a single class and restricting direct access to the data.

Java achieves encapsulation mainly by using access modifiers and getter/setter methods.

- private → prevents direct access to data from outside the class.
- public getter → allows reading the private data
- public setter → allows controlled modification of the private data.

Example:

```
class student {  
    private string name; // Encapsulation  
                           data  
    public string getName () {  
        return name;  
    }  
}
```

2

```
public void setName (string name) {  
    this.name = name;  
}  
}
```

Here, name cannot be accessed directly. It can only be accessed through setName() and getName().

Real-life analogy:

An ATM is an example of encapsulation. You cannot directly access the money inside the bank's system. You interact with it through controlled operations such as deposit, withdraw and balance inquiry.

Question - 28

What is Polymorphism? Distinguish compile-time (static) vs run-time (dynamic) polymorphism with one example each.

Answer:

Polymorphism means "many forms". In java, the same method name can perform different operations depending on the situation.

Compile-time	Run-time
Method Overloading	Method Overriding
Static binding	Dynamic binding
Decided during compilation	Decided during execution
Same class is commonly used	Inheritance is required

□ Method Overloading:

Method overloading means having multiple methods with the same name but different parameters in the same class.

Parameters can differ by:

- Number of parameter.
- Type of Parameters.
- Order of Parameters.

Example:

```
class Calculator {  
    int add (int a, int b) {  
        return a+b;  
    }  
    int add (int a, int b, int c) {  
        return a+b+c;  
    }  
}
```

Both methods are named add(), but their parameter lists are different.

Therefore, this is Compile-Time polymorphism.

□ Method Overriding:

Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

Example

```
class Animal {  
    void sound () {  
        System.out.println("Animal makes sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound () {  
        System.out.println("Dog barks");  
    }  
}
```

Here, Dog overrides the sound() method of Animal. When a parent reference refers to a child object, Java decides which method to execute at runtime.

Therefore, it is called Runtime Polymorphism.

Practical:

Write a BankAccount class demonstrating encapsulation (private balance + getBalance() / deposit()) and polymorphism (overload deposit(double) / deposit(double, String remarks). Include a main class calling both overloads.

Answer:

BankAccount.java

This class demonstrates encapsulation using a private balance and getBalance(), and compile-time polymorphism using overloaded deposit() methods

```
class BankAccount {
    // Encapsulation: private data
    private double balance;
    // constructor
    BankAccount (double balance) {
        this.balance = balance;
    }
    // Getter method
    public double getBalance() {
        return balance;
    }
}
```

```

// method 1: deposit(double)
public void deposit(double amount) {
    balance += amount;
    System.out.println("Deposit: " + amount);
}

// method 2: deposit(double, string)
public void deposit(double amount, string
                    remarks) {
    balance += amount;
    System.out.println("Deposit: " + amount);
    System.out.println("Remarks: " + remarks);
}
}

```

Main.java

```

public class main {
    public static void main (String[] args) {
        // creating BankAccount object
        BankAccount account = new BankAccount(5000);
        // calling first overloaded method
        account.deposit(1000);
    }
}

```


// calling second overloaded method

account.deposit(2000, "Salary Deposit");

// Getting balance using getter

System.out.println("Current Balance: "
+ account.getBalance());

}

Output:

Deposited: 1000.0

Deposited: 2000.0

Remarks: Salary Deposit

Current Balance: 8000.0

Encapsulation:

private double balance;

The balance variable cannot be accessed directly from main.

Getter:

getBalance();

provides controlled access to the balance.

Polymorphism / Method Overloading:

deposit(double)

deposit(double, String)

Both methods have the same name but different parameter lists. Therefore, this is compile-time (static) polymorphism.

□ L2 - Method Overloading vs Overriding (Early vs Late Binding)

□ Question - 1:

Compare overloading vs overriding: definition, class involved, parameters, return type, binding time

Answer:

Feature	Method Overloading	Method Overriding
Definition	Same method name with different parameters	Child class provides a new implementation of parent method.
Class involved	Usually same class	Parent and child classes
Parameters	Must be different	Must be the same
Return type	May be same or different but return type alone cannot overload.	Same or covariant return type
Binding time	Compile time	Run time
Polymorphism	Compile-time polymorphism	Run-time Polymorphism

Question-28

Explain early binding vs late binding. why is overriding resolved at run time and overloading at compile time?

Answer:

Early Binding

Early binding means the method call is resolved at compile time.

Method overloading uses early binding because the compiler can determine which method to call by checking the method name and parameter list.

Example:

```
void describe (string name) { }  
void describe (string name, int age) { }
```

The compiler knows which describe() method to call from the arguments.

Late Binding

Late binding means the method call is resolved at run time.

Method overriding uses late binding because the actual object type determines which overridden method will execute.

Example:

```
shape s = new Circle();  
s.area();
```

Although the reference is of type shape, the actual object is Circle, so Circle's area() method is executed.

Why?

- Overloading → compiler knows the parameter list → compile time.
- Overriding → actual object is determined at runtime → run time.

Practical:

Shape with area(), overridden by Circle and Rectangle (late Binding via shape reference).
Add overloaded describe(string) / describe(string, int) in Shape (early binding). Show sample output.

Answers:

The program below demonstrates:

1. method overriding using Shape, Circle and Rectangle
2. Late binding using a Shape reference.
3. method overloading using describe(string) and describe(string, int).
4. Early binding for the overloaded methods.

```
class Shape {  
    // method to be overridden  
    public void area() {  
        System.out.println("Area of Shape");  
    }  
    // overloaded method 1  
    public void describe(string name) {  
        System.out.println("Shape: " + name);  
    }  
}
```


// Overloaded method ?

```
public void describe (String name, int sides) {
    System.out.println ("Shape: " + name);
    System.out.println ("Number of sides: " + sides);
}
```

// Child class: Circle

```
class Circle extends Shape {
```

```
@Override
```

```
public void area () {
```

```
    double radius = 5;
```

```
    double result = Math.PI * radius * radius;
```

```
    System.out.println ("Area of circle: " +
        result);
}
```

// Child class: Rectangle

```
@Override
```

```
public void area () {
```

```
    double length = 10;
```

```
    double width = 5;
```

```
    double result = length * width;
```

```
    System.out.println ("Area of Rectangle: " +
        result);
}
```

// main class

```
public class main {
```

```
    public static void main (String [] args) {
```

// Late Binding / Run-time Polymorphism

```
    Shape s1 = new Circle ();
```

```
    Shape s2 = new Rectangle ();
```

```
    s1.area ();
```

```
    s2.area ();
```

```
    System.out.println ();
```

// Early Binding / method Overloading

```
    Shape s3 = new Shape ();
```

```
    s3.describe ("Circle");
```

```
    s3.describe ("Rectangle", 4);
```

```
    }
}
```

Output:

Area of Circle: 78.53981633974483

Area of Rectangle: 50.0

Shape: Circle

Shape: Rectangle

Number of sides: 4

Q1-3 - Abstract class vs Interface

Question-1:

Define abstract class & interface. List > 4 structural difference (fields, constructors, method bodies, multiple inheritance).

Answer:

Abstract class is a class declared with the abstract keyword. It can contain both abstract methods (without body) and concrete methods (with body). It is used when related classes share common state and behavior.

An interface is a blueprint that defines a set of methods that implementing classes must provide. It is mainly used to define capabilities or contracts.

Feature	Abstract class	Interface
Fields	can have instance variables, static variables, constants	Fields are implicitly public, static, final
Constructor	can have constructors	cannot have constructor
Method Bodies	can contain abstract and concrete methods	can contain abstract methods and also default/static methods

Inheritance	A class can extend ^{only one} class	A class can implement multiple interfaces
keyword	extends	implements
Purpose	Represents a common "is-a" relationship	Represents a "can-do" capability

Question-2:

When should you choose an abstract class vs an interface? Give two real-world scenarios ("is-a" with shared code vs "can-do" capability).

Answer:

Choose an Abstract class when:

- classes have a strong "is-a" relationship.
- They need to share common fields/state.
- They need to share common implemented methods

Real world example:

Car, Bus and Truck are all vehicles. They can share fields such as speed and common methods such as startEngine(). Therefore; an abstract Vehicle class is suitable.

Choose an Interface when:

- You want to define a "can-do" capability.
- Unrelated classes may need the same capability.
- Multiple inheritance of behavior/contracts is required.

Real-world example:

A car, House or Business may all be insurable. They are not necessarily related by inheritance they can all perform the capability calculatePremium(). Therefore, an Insurable interface is suitable.

Practical:

Vehicle (abstract, startEngine() implemented + abstract fuelType() and Insurable (interface calculatePremium()). Car extends Vehicle and implements Insurable. Comment on why each construct was chosen.

Answer:

```
// abstract class
abstract class Vehicle {
    // common method with implementation
    public void startEngine() {
        System.out.println("Vehicle engine started.");
    }

    // Abstract method
    public abstract String fuelType();
}
```

4-
// Interface

```
interface Insurable{
```

```
    // Abstract method
```

```
    double calculatePremium();
```

```
}
```

```
// car extends Vehicle and implements Insurable  
class Car extends Vehicle implements Insurable{
```

```
    private String model;
```

```
    private double price;
```

```
    // constructor
```

```
    public Car (String model, double price){
```

```
        this.model = model;
```

```
        this.price = price;
```

```
}
```

```
// Implementing abstract method from Vehicle
```

```
@Override
```

```
    public String fuelType() {
```

```
        return "Petrol";
```

```
}
```

```
// Implementing method from Insurable
```

```
@Override
```

```
    public double calculatePremium() {
```

```
        return price * 0.05;
```

```
}
```

// Display information

```
public void displayInfo () {
```

```
System.out.println ("Car model: " + model);
```

```
System.out.println ("Car price: " + price);
```

```
System.out.println ("Fuel Type: " + fuelType);
```

```
System.out.println ("Insurance Premium: " + calculatePremium());
```

```
}
```

// main class

```
public class main {
```

```
public static void main (String[] args) {
```

```
Car car = new Car ("Toyota Corolla", 2000000);
```

```
car.startEngine();
```

```
car.displayInfo();
```

```
}
```

Output: vehicle engine started

Car model: Toyota Corolla

Car price: 2000000.0

Fuel Type: Petrol

Insurance Premium: 100000.0

Why Each Construct was Chosen

1. Vehicle - Abstract Class;
used because car is a type of vehicle and can share common methods.
2. startEngine() - Concrete method.
Provides a common implementation that all vehicles can use.
3. FuelType() - Abstract method.
Different vehicles can have different fuel types, so subclasses define it.
4. Insurable - Interface
Used to represent a capability - vehicles can be insured.
5. calculatePremium() - Interface method.
Requires every insurable class to define its own premium calculation.
6. car extends Vehicle:
Used because a car is a vehicle (is-a relationship)
7. car implements Insurable:
Used because a car can be insured (can-do relationship).

Q4 - Collection Framework

Question-1:

Compare ArrayList, Vector, LinkedList; underlying structure, synchronization, insert/delete vs Random-access performance.

Answer:

Feature	ArrayList	Vector	LinkedList
Structure	Dynamic array	Dynamic array	Doubly Linked list
Synchronization	Not Synchronization	Synchronized	Not Synchronized
Random Access	Fast - $O(1)$	Fast - $O(1)$	Slow - $O(n)$
Insertion/Deletion	Slow in middle - $O(n)$	Slow in middle - $O(n)$	Fast at known position - $O(1)$
Performance	Generally	Slower due to synchronization	Good for frequent insert/delete.

Question-2:

Explain set and its implementations HashSet, LinkedHashSet, TreeSet. How does TreeSet maintain order?

Answer:

A set is a collection that does not allow duplicate elements.

Q4 - Collection Framework

Question - 1:

Compare ArrayList, Vector, LinkedList: underlying structure, synchronization, insert/delete vs Random-access performance.

Answer:

Feature	ArrayList	Vector	LinkedList
Structure	Dynamic array	Dynamic array	Doubly Linked list
Synchronization	Not Synchronization	Synchronized	Not Synchronized
Random Access	Fast - $O(1)$	Fast - $O(1)$	Slow - $O(n)$
Insertion/Deletion	Slow in middle - $O(n)$	Slow in middle - $O(n)$	Fast at known position - $O(1)$
Performance	Generally	Slower due to synchronization	Good for frequent insert/delete.

Question - 2:

Explain set and its implementations HashSet, LinkedHashSet, TreeSet. How does TreeSet maintain order?

Answer:

A set is a collection that does not allow duplicate elements.

Practical:

store 5 student names in an ArrayList (print with enhanced for-loop) and in a TreeSet (print, observe sorting). comment on ordering difference.

Answer:

```
import java.util.ArrayList;
import java.util.TreeSet;

public class main {
    public static void main (String[] args) {
        // store 5 student names in ArrayList.
        ArrayList<String> students = new ArrayList<>();
        students.add ("Rahim");
        students.add ("karim");
        students.add ("Nusrat");
        students.add ("Alif");
        students.add ("sadia");

        // print ArrayList using enhanced for-loop
        System.out.println ("ArrayList:");
        for (String name : students) {
            System.out.println (name);
        }
    }
}
```


// store same names in TreeSet

```
TreeSet<String> sortedStudents = new TreeSet<>();
```

```
sortedStudents.add("Rahim");
```

```
sortedStudents.add("karim");
```

```
sortedStudents.add("Nusrat");
```

```
sortedStudents.add("Alif");
```

```
sortedStudents.add("sadia");
```

// Print TreeSet

```
System.out.println("\nTreeSet:");
```

```
for (String name: sortedStudents) {
```

```
    System.out.println(name);
```

```
}
```

```
}
```

```
}
```

Output:

ArrayList:

Rahim

karim

Nusrat

Alif

sadia

TreeSet:

Alif

karim

Nusrat

Rahim

sadia

Comment on Ordering Difference

- ArrayList maintains the insertion order of elements.
- TreeSet automatically maintains elements in sorted order
- Therefore, the same names can appear in different orders in ArrayList and TreeSet.

DL5- Multithreading & custom Exception Handling

Question-1:

List ≥ 3 ways to implement multithreading in Java
(extends Thread, implements Runnable,
ExecutorService/callable) which is preferred
and why?

Answer:

There are several ways to implement multi-threading in Java:

1. By extending Thread

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}
```

2. By implementing Runnable

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}
```

3. Using `ExecutorService`

```
ExecutorService executor = Executors.newFixedThreadPool(2);
```

```
executor.submit(() -> System.out.println  
    ("Task running"));
```

```
executor.shutdown();
```

4. Using `Callable`

`Callable` is similar to `Runnable`, but it can return a value and throw exceptions.

Which is preferred and why?

`Runnable` / `ExecutorService` is generally preferred over extending `Thread`.

Reasons:

- Java supports single inheritance, so extending `Thread` prevents the class from extending another class.
- `Runnable` separates the task from the thread.
- `ExecutorService` provides better thread management and thread pools.
- It is more scalable and suitable for large applications.

Practical - 1:

Two threads printing 1-5 concurrently - one via

extends Thread, one via implements

Runnable. Show how both are started from

main().

Answer:

// Thread using extends Thread

class NumberThread extends Thread {

public void run() {

for (int i=1; i<=5; i++) {

System.out.println("Thread class: " + i);

} try {

Thread.sleep(100);

} catch (InterruptedException e) {

System.out.println(e.getMessage());

}}}}

// Thread using implements Runnable

class NumberRunnable implements Runnable {

public void run() {

for (int i=1; i<=5; i++) {

System.out.println("Runnable: " + i);

} try {

Thread.sleep(100);

}

4

```
catch (InterruptedException e) {  
    System.out.println (e.getMessage());  
}}}}
```

// main class

```
public class main {  
    public static void main (String[] args) {  
        // creating Thread object  
        NumberRunnable runnable = new  
            NumberRunnable();  
        Thread t2 = new Thread (runnable);  
        // starting both threads  
        t1.start();  
        t2.start();  
    }  
}
```

Output:

```
Thread class : 1  
Runnable : 1  
Thread class : 2  
Runnable : 2  
Runnable : 3  
Thread class : 3  
Thread class : 4  
Runnable : 4  
Runnable : 5  
Thread class : 5
```

Use start(), not run(), to actually create and execute a separate thread.

Practical-2:

Custom checked exception InvalidRadiusException.
 Circle constructor throws it if radius < 0. Main
 reads radius, handles via try-catch, else
 prints area.

Answer:

```
import java.util.Scanner;
```

```
// custom checked Exception
```

```
class InvalidRadiusException extends Exception{
```

```
    public InvalidRadiusException (String message){
        super (message);
```

```
    }
```

```
// Circle class
```

```
class Circle{
```

```
    private double radius;
```

```
// constructor
```

```
    public Circle (double radius) throws
        InvalidRadiusException{
```

```
        if (radius < 0) {
```

```
            throw new InvalidRadiusException{
```

```
                "Radius cannot be negative"
```

```
            };
```

```
        }
```

6

```

        this.radius = radius;
    }
    // calculate area
    public double getArea() {
        return Math.PI * radius * radius;
    }
}

```

// main class

```

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter radius: ");
        double radius = input.nextDouble();

        try {
            Circle circle = new Circle(radius);
            System.out.println("Area = " + circle.getArea());
        } catch (InvalidRadiusException e) {
            System.out.println("Exception: " + e.getMessage());
        }

        input.close();
    }
}

```

Example 1: Valid Radius

Enter radius: 5

Area = 78.53981633

Example 2: Invalid Radius

Enter radius: -3

Exception: Radius cannot be negative!

Q 16- JDBC with MySQL/Oracle (mve Pattern)

Question-1:

steps to connect java to MySQL/Oracle via JDBC.
Name key classes (DriverManager, Connection, PreparedStatement, ResultSet).

Answer:

JDBC (Java Database Connectivity) is an API used to connect Java applications with databases such as MySQL and Oracle.

Steps:

1. Load / Register the JDBC Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

2. Establish Connection

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/studentdb",
    "root",
    "password"
);
```

3. Create PreparedStatement

```
PreparedStatement ps =
    con.prepareStatement("INSERT INTO student
    VALUES (?, ?)");
```

4. Set Parameters and Execute Query

```
ps.setInt(1, 101);
```

```
ps.setString(2, "Ali");
```

```
ps.executeUpdate();
```

5. Process ResultSet for SELECT queries

```
ResultSet rs = ps.executeQuery();
```

```
while (rs.next()) {
```

```
System.out.println(rs.getString("name"));
```

```
}
```

6. Close Resources

```
rs.close();
```

```
ps.close();
```

```
con.close();
```

• Key JDBC Classes

Class/Interface	Purpose
DriverManager	Establishes database connection
Connection	Represents connection with database
PreparedStatement	Executes parameterized queries
ResultSet	Stores results returned by SELECT queries.

Question - 2%

Explain the MVC Pattern and how a JDBC app's classes map to model/view/controller.

Answer:

MVC stands for

- M - Model
- V - View
- C - Controller

MVC separates an application into three major components,

1. Model

The model represents the application's data and business objects.

Example:

```
class student {  
    int id;  
    String name;  
}
```

In a JDBC application, the model may contain classes such as student.

2. View

The view is responsible for interacting with the user and displaying output.

Example: `Scanner input = new Scanner(System.in);
System.out.println("Enter student name:");`

Here, main can act as the view.

3. Controller

The controller receives input from the View and controls the application's operations.

In a JDBC application, a DAO (Data Access Object) can perform database operations.

Example:

```
studentDAO.insert(student);
```

Advantages of MVC

- Separates responsibilities
- Makes code easier to maintain
- Makes testing easier.
- Allows database code to remain separate from user-interface code.
- Improves code organization.

Practical:

MVC-style code for student: Model (student), Controller / DAO (studentDAO.insert()) using Prepared Statement, View (main) collecting input and calling the controller.

Answer:

Database Table

First, create a MySQL database and table:

```
CREATE DATABASE studentdb;
USE studentdb;

CREATE TABLE student (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
);
```

1. Model - student.java

```
// Model
class student {
    private int id;
    private String name;
    private String email;
    public student (int id, String name, String email) {
        this.id = id;
        this.name = name;
    }
}
```

```

        this.email = email;
    }
    public int getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public String getEmail() {
        return email;
    }
}

```

2. DAO / Controller — StudentDAO.java

```
import java.sql.*;
```

```
class StudentDAO {
```

```
    public void insert(Student s) {
```

```
        String url = "jdbc:mysql://localhost:3306/";
        String dbName = "studentdb";
```

```
        String user = "root";
```

```
        String pass = "password";
```

```
        String sql = "INSERT INTO student VALUES";
        sql += "(?, ?, ?)";
```

```
        try {
```

```
            Connection con =
```

```
            DriverManager.getConnection(url, user, pass);
```

```

PreparedStatement ps = con.prepareStatement(sql);
ps.setInt (1, s.getId());
ps.setString (2, s.getName());
ps.setString (3, s.getEmail());
ps.executeUpdate();
System.out.println("Student inserted
                    successfully.");
con.close();
} catch (Exception e) {
    System.out.println(e);
} } }

```

3. View - main.java

```
import java.util.Scanner;
```

```
// view
```

```

public class main {
    public static void main (String [] args) {
        Scanner input = new Scanner (System.in);
        // collect input
        System.out.print ("Enter Student ID: ");
        int id = input.nextInt();
        input.nextLine();
    }
}

```

```

System.out.print ("Enter student Name: ");
String name = input.nextLine();
System.out.print ("Enter student Email: ");

// create model object
student student = new student (id, name,
                                email);

// Create DAO / Controller Object
studentDAO dao = new studentDAO();

// call controller / DAO
dao.insert (student);
input.close();
} }

```

Output:

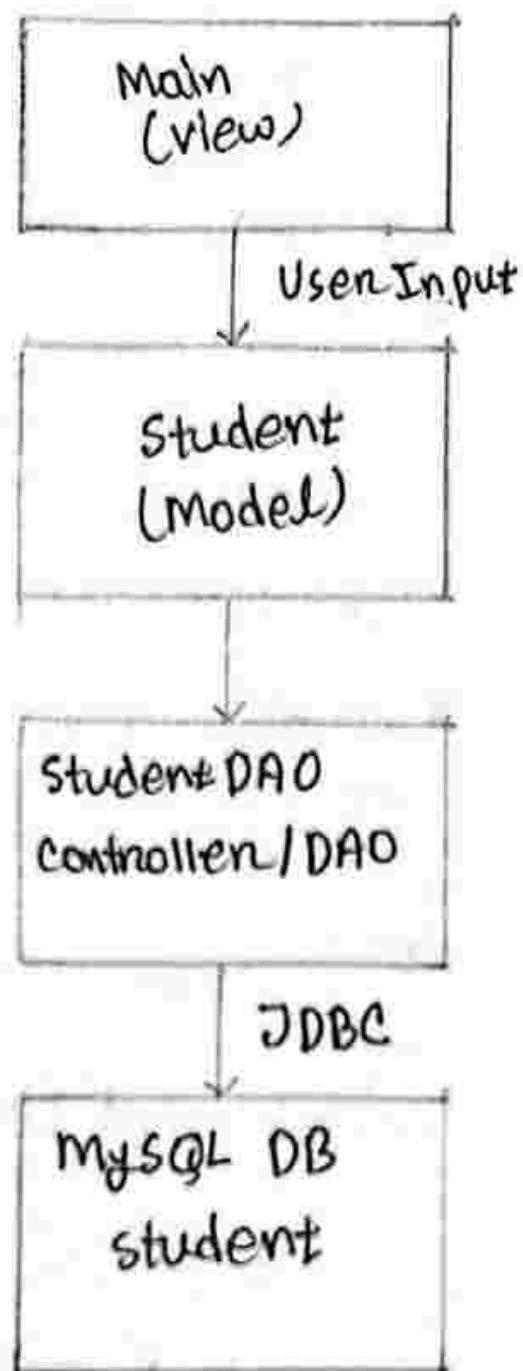
Enter student ID: 101

Enter student Name: Alif

Enter student Email: alif@gmail.com

student inserted successfully.

Complete MVC Flow



Q17-JavaFX-House Loan Calculator

Question-1:

Explain the JavaFX application structure (Application, stage, scene, Gridpane/VBox). Role of start()??

Answers:

JavaFX application mainly consists of Application, stage, scene, and layout.

- Application: main class of a JavaFX program.
- stage: The main window.
- scene: Contains all UI components.
- Gridpane: Arranges components in rows and columns.
- VBox: Arranges components vertically.
- start(): The main method of JavaFX application. It is called automatically when the application starts.

```
public void start (stage stage) {  
    Gridpane grid = new Gridpane();  
    scene scene = new scene (grid, 400, 300);  
    stage.setScene (scene);  
    stage.show();  
}
```

Question-2:

Design the GridPane layout: Loan Amount, Annual Rate, Number of years inputs + "Calculate" button + result labels.

Answer:

The layout contains Loan Amount, Annual Rate, Number of years Calculate button and result labels.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;
```

```
public class LoanCalculator extends Application {
```

```
    public void start (Stage stage) {
```

```
        GridPane grid = new GridPane();
```

```
        TextField loan = new TextField();
```

```
        TextField rate = new TextField();
```

```
        TextField years = new TextField();
```

```
        Button calculate = new Button("Calculate");
```

```
        Label monthly = new Label("month/y Installment:");
```

```
        Label total = new Label("Total Payments:");
```

```
        Label difference = new Label("Difference:");
```

```

grid.add(new Label("Loan Amount: "), 0, 0);
grid.add(loan, 1, 0);
grid.add(new Label("Annual Rate: "), 0, 1);
grid.add(rate, 1, 1);
grid.add(new Label("Years: "), 0, 2);
grid.add(years, 1, 2);
grid.add(calculate, 1, 3);
grid.add(monthly, 0, 4);
grid.add(total, 0, 5);
grid.add(difference, 0, 6);
stage.setScene(new Scene(grid, 400, 300));
stage.setTitle("House loan calculator");
stage.show();
}
public static void main (String[] args) {
    launch (args);
}
}

```


Practical:

Event handler computing monthly Installment (amortization formula), Total Payment and Difference (Total Payment - Loan Amount).

Answer:

The amortization formula is:

$$m = \frac{P \times r(1+rn)^n}{(1+rn)^n - 1}$$

When P = Loan Amount, r = Monthly Interest Rate,
And n = Number of monthly Payments.

calculate. setOnAction($e \rightarrow$ {

double P = Double.parseDouble(loan.getText());

double r = Double.parseDouble($\text{rate.getText()}/1200$);

int n = Integer.parseInt($\text{years.getText()} * 12$);

double m = $P * r * \text{math.pow}(1+r, n) /$
 $(\text{math.pow}(1+r, n) - 1);$

double T = $m * n$;

double D = $T - P$;

monthly.setText("monthly Installment: " + m);

total.setText("Total Payment: " + T);

difference.setText("Difference: " + D);

});

Input:

Loan Amount = 100000

Annual Rate = 5%

Years = 10

Output:

Monthly Installment: 1060.66

Total Payment: 127278.62

Difference: 27278.62

Q18 - Socket Programming & Java RMI (Chat System)

Question-1:

Compare socket programming vs Java RMI, when to prefer each?

Answer:

Socket Programming	Java RMI
Direct network communication	calls methods of remote objects
uses socket and serversocket	Uses remote objects / interface
more control over communication	Easier for Java applications
Good for chat systems	Good for distributed Java programs

Socket is preferred for simple client-server communication such as chat systems. RMI is preferred when Java programs needed to call methods of objects located on another computer.

Practical:

server code using serverSocket: accept connection, read via BufferedReader, reply via PrintWriter.

Answer:

```
import java.io.*;
import java.net.*;

public class server {
    public static void main (String[] args) throws
        Exception {
        ServerSocket ss = new ServerSocket (5000);
        Socket s = ss.accept();
        BufferedReader in = new BufferedReader (
            new InputStreamReader (s.getInputStream()));
        PrintWriter out = new PrintWriter (
            s.getOutputStream(), true);
        String msg = in.readLine();
        System.out.println ("Client: " + msg);
        out.println ("Hello Client! message received.");
        s.close();
        ss.close();
    }
}
```


main points:

- ServerSocket accepts the connection.
- BufferedReader reads the client message.
- PrintWriter sends the reply.

Practical:

Client code using socket: connect to server, send typed message, print server reply.

Answer:

```
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) throws
        Exception {
        Socket s = new Socket("localhost", 5000);
        BufferedReader in = new BufferedReader(
            new InputStreamReader(
                s.getInputStream()
            ));
        PrintWriter out = new PrintWriter(
            s.getOutputStream(), true);
        out.println("Hello Server!");
        System.out.println("server: " +
            in.readLine());
        s.close();
    }
}
```

Output:

Client: Hello Server!

Server: Hello Client! message received.

□ L9 - servlet + JSP + JDBC CRUD (Student Records)

Question - 1:

Steps to set up student db / students table project with servlet + JSP + JDBC (DB setup, mapping, connection strings).

Answer:

To create a student Records projects using
servlet + JSP + JDBC;

1. create a database named of student db
2. create a students table.

```
CREATE DATABASE student_db;
```

```
USE student_db;
```

```
CREATE TABLE students(  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    gpa DOUBLE  
);
```

3. Add the MySQL JDBC driver to the project
4. Configure the servlet mapping in web.xml or using @webServlet.
5. Use a JDBC connection string.

2
string url = "jdbc:mysql://localhost:3306/
studentdb";

string user = "root";

string password = "";

6. Use servlet for processing requests,
JSP for displaying data and JDBC for
database connection.

Question-2:

Practical:

servlet doPost(): read ID/Name/CGPA from form,
insert via PreparedStatement, with exception
handling.

Answer:

The servlet reads ID, Name and CGPA from
the form and inserts them using
PreparedStatement.

```
import java.io.*;
```

```
import java.sql.*; import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```
import javax.servlet.annotation.WebServlet;
```

```
@WebServlet("/addstudent")
```

```
public class StudentServlet extends  
HttpServlet {
```



```

protected void doPost(HttpServletRequest req,
                        HttpServletResponse res)
                        throws ServletException,
                        IOException {
    try {
        int id = Integer.parseInt(req.getParameter(
            "id"));
        String name = req.getParameter("name");
        double gpa = Double.parseDouble(
            req.getParameter("gpa"));
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/student_db",
            "root", "");
        String sql = "INSERT INTO student VALUES
            (?, ?, ?)";
        PreparedStatement ps = con.prepareStatement(
            sql);
        ps.setInt(1, id);
        ps.setString(2, name);
        ps.setDouble(3, gpa);
        ps.executeUpdate();
    }
}

```

4

```
res.getWriter().println("student Added  
successfully!");
```

```
con.close();
```

```
} catch (Exception e) {  
    res.getWriter().println("Error: "  
        + e.getMessage());  
} }
```

It safely handles parameters and helps prevent SQL injection.

Practical 2:

JSP page: connect to studentdb, SELECT from students, display as HTML table.

Answers:

The JSP connects to studentdb, retrieves records using SELECT and displays them in an HTML table.

<?@ page import = "java.sql.*" ?>

<html>

<body>

<h2>student Records</h2>

<table border="1">

<tr>

<th>ID</th>

<th>Name</th>

<th>CGPA</th>

</th>

<?>

try {

connection con = DriverManager.
getConnection(

"jdbc:mysql://localhost:3306/student-db",

"root", "");

Statement st = con.createStatement();

ResultSet rs = st.executeQuery(

"SELECT * FROM 'student'");

while (rs.next()) {

?>

<tr> .

<td><%=rs.getInt("id")%></td>

<td><%=rs.getInt("name")%></td>

<td><%=rs.getDouble("GPA")%></td>

</tr>

<%=

}

con.close();

Output%

ID	Name	GPA
1	Aliif	3.75
2	Rahim	3.50